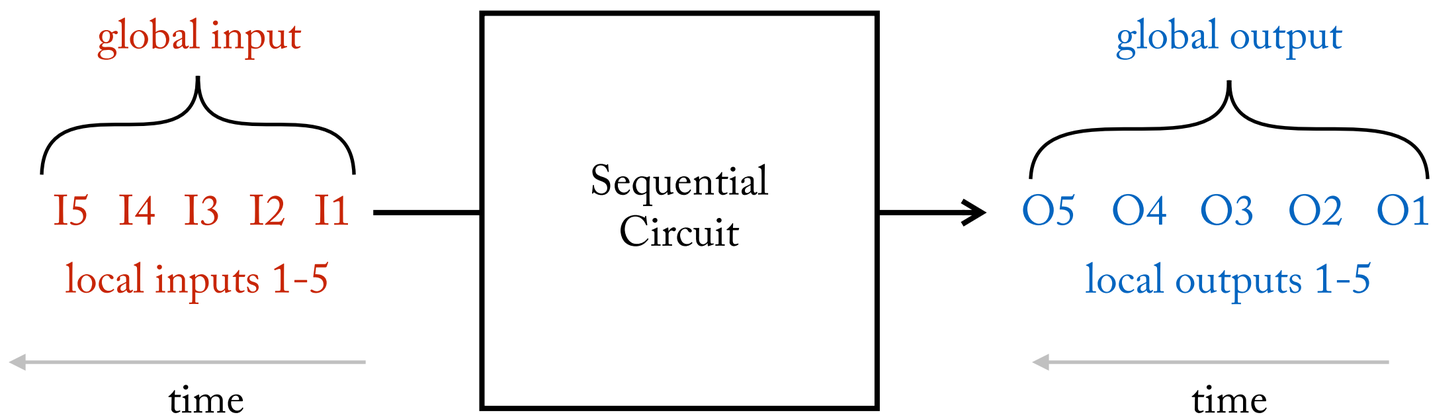
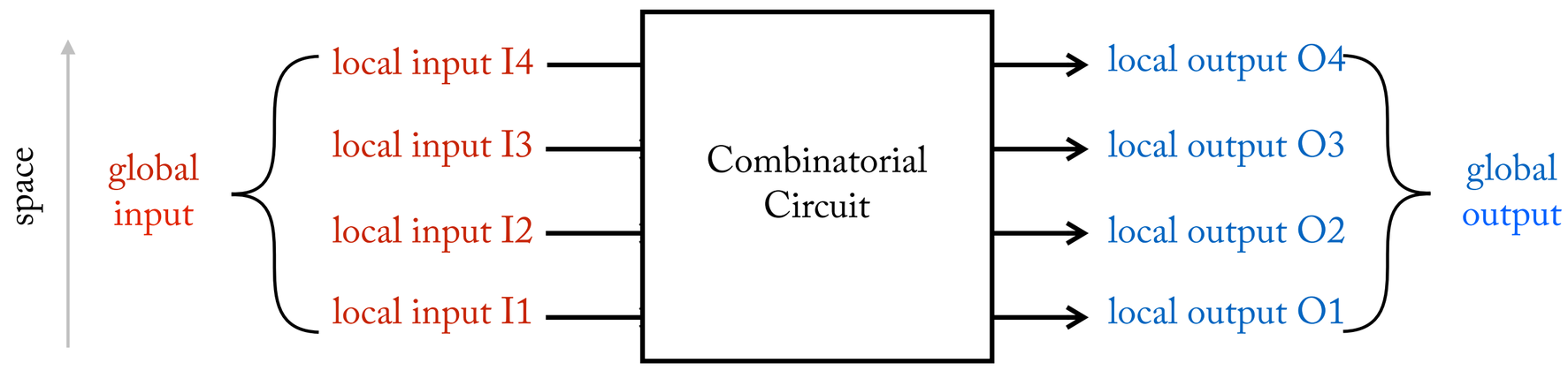
19. Computing in Time

Inputs and Outputs

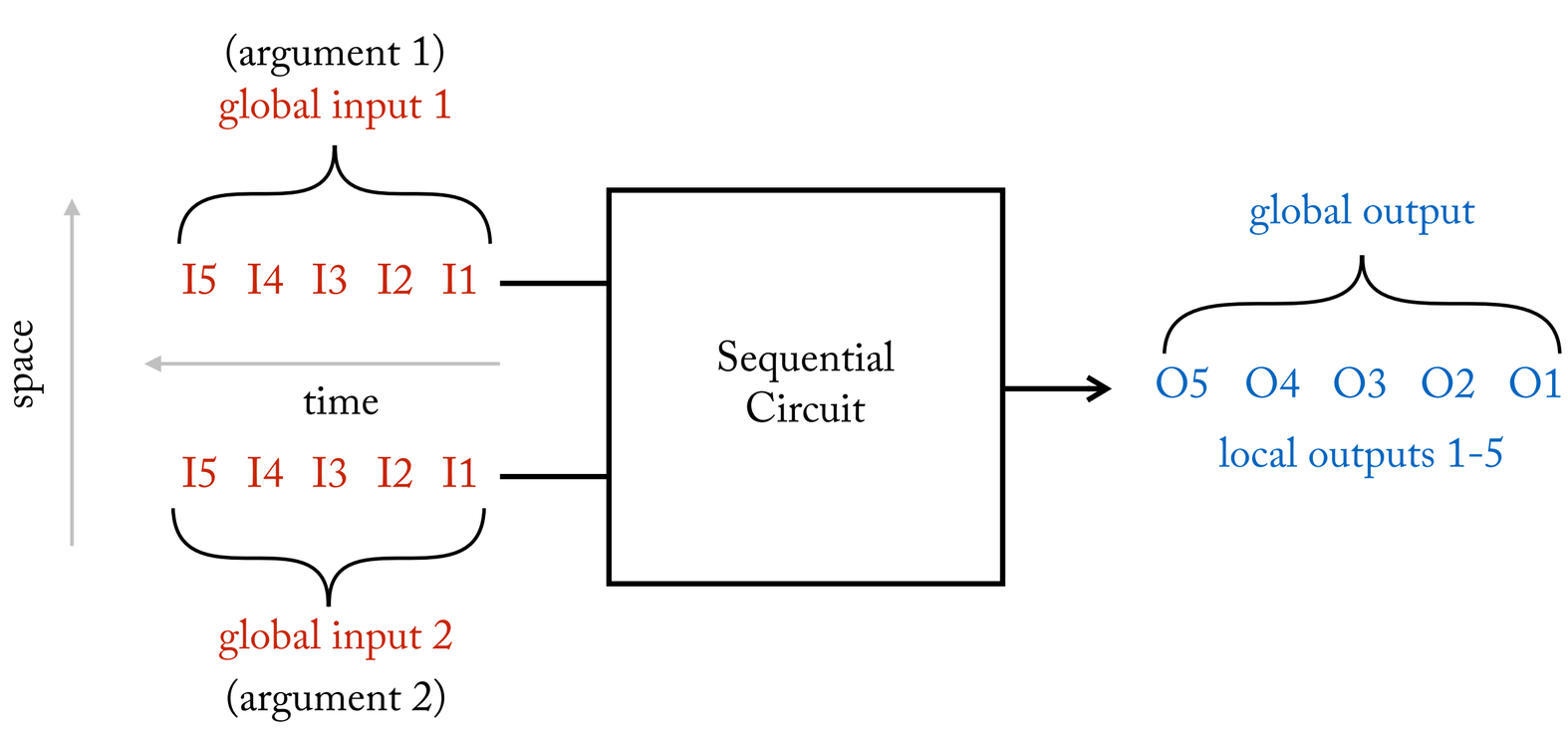
We now introduce the concept of **input at a time**, as well as the overall **input over time**. The input at a time corresponds to an SC's **local input**, the input over time corresponds to an SC's **global input**. So, for SCs, arguments and values are represented by complex symbols over time.



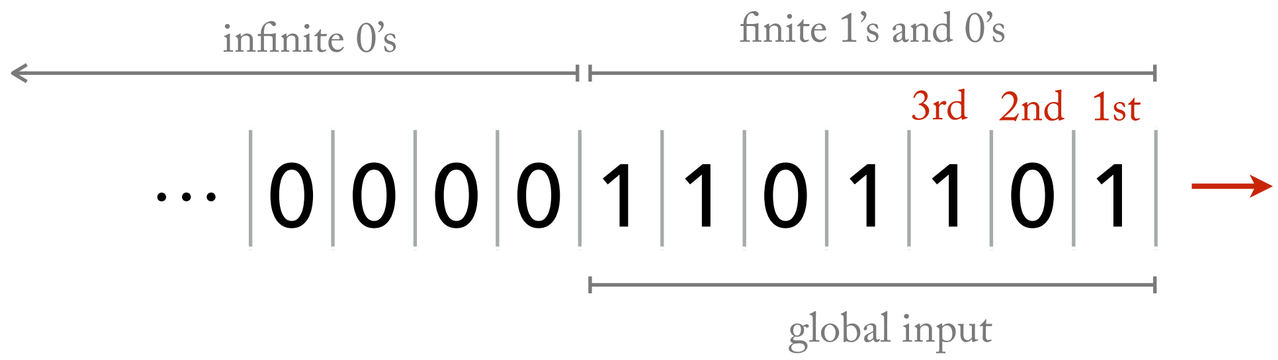


SCs do with time what CCs do with space. The major difference is that space is finite, but time is (for all practical purposes) unlimited.

By the way: SCs can have more than one global input. When performing computation, each argument-place in the computed function corresponds to a distinct global input in the circuit.

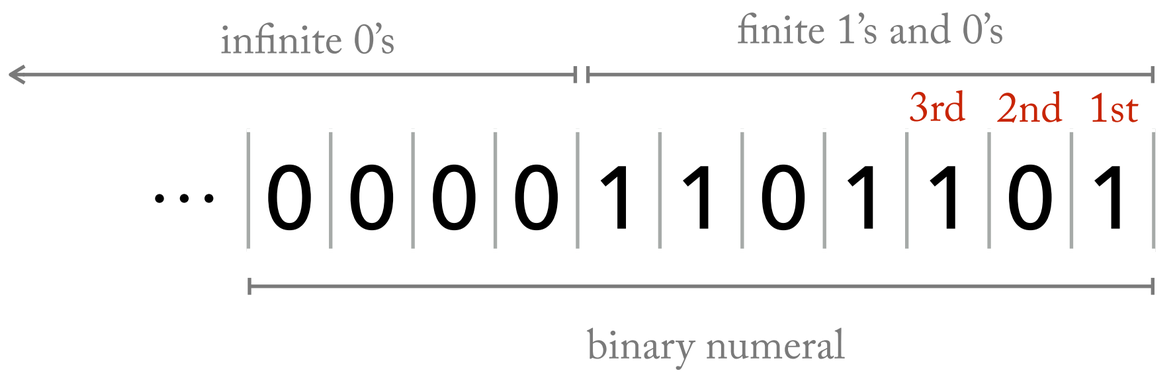


Computing with Sequential Circuits



On any one occasion, we'll imagine that an SC goes on forever. It receives an infinite number of digits over an infinite amount of time. We'll assume that the input always begins with a finite number of 1's and 0's followed by an infinite number of 0's.

When an SC computes a numerical function, the global inputs and outputs are those finite sequences of local inputs/outputs at the point when all remaining local inputs *and* outputs are 0's.



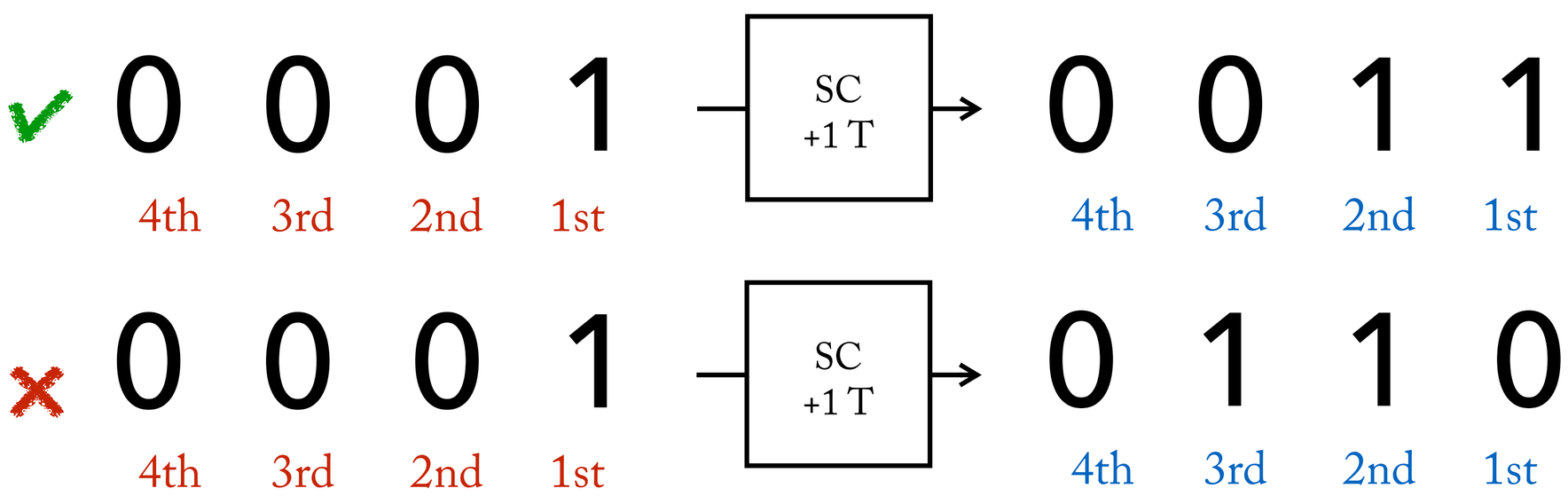
Numerals in Tally:

- Entered as a finite number of consecutive 1's followed by an infinite number of 0's. E.g. 00001, 00011, 00111, 01111, etc.
- 0 will count as a numeral in Tally (representing zero).

Numerals in Binary:

- Made up of a finite number of 0's and 1's followed by an infinite number of 0's.

Computing a function with SCs is just like computing a function with CCs. Only now, the inputs arrive in temporal order, rather than spatial order.



+1 T: One More Time with Feeling…

Here is the temporal input-output table for +1 T. It's a temporal version of the +1 T CC. It always produces a 1 first and then delays every other bit by 1 cycle.

+1 T where x=0

t5	t4	t3	t2	t1	
…	0	0	0	0	IN
…	0	0	0	1	OUT

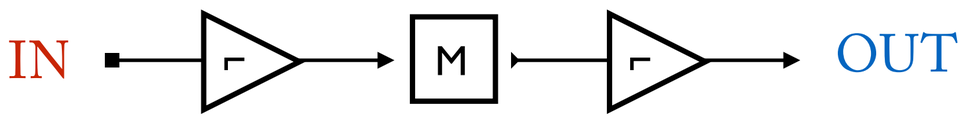
+1 T where x=2

t5	t4	t3	t2	t1	
…	0	0	1	1	IN
…	0	1	1	1	OUT

We can always delay a bit by 1 cycle by introducing a memory in the middle of a wire.



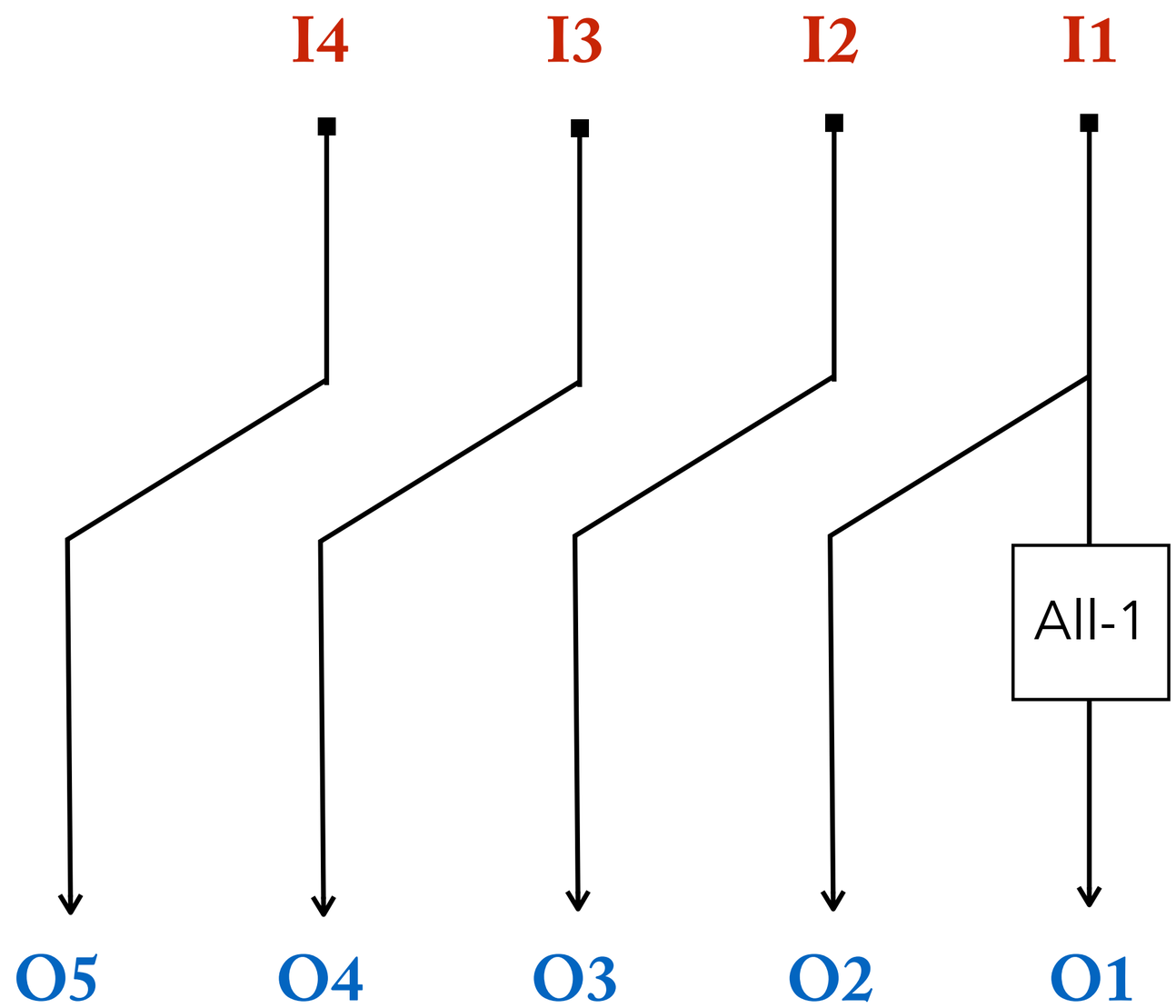
Of course for +1 T, we need the first bit to be a 1. So then it would make sense to use a **1-first memory**, and use *that* to delay the incoming bit.



Voila! +1 T SC

If you squint in just the right way, you should be able to see that our solution for SC +1 T is the same as our solution to CC +1 T.

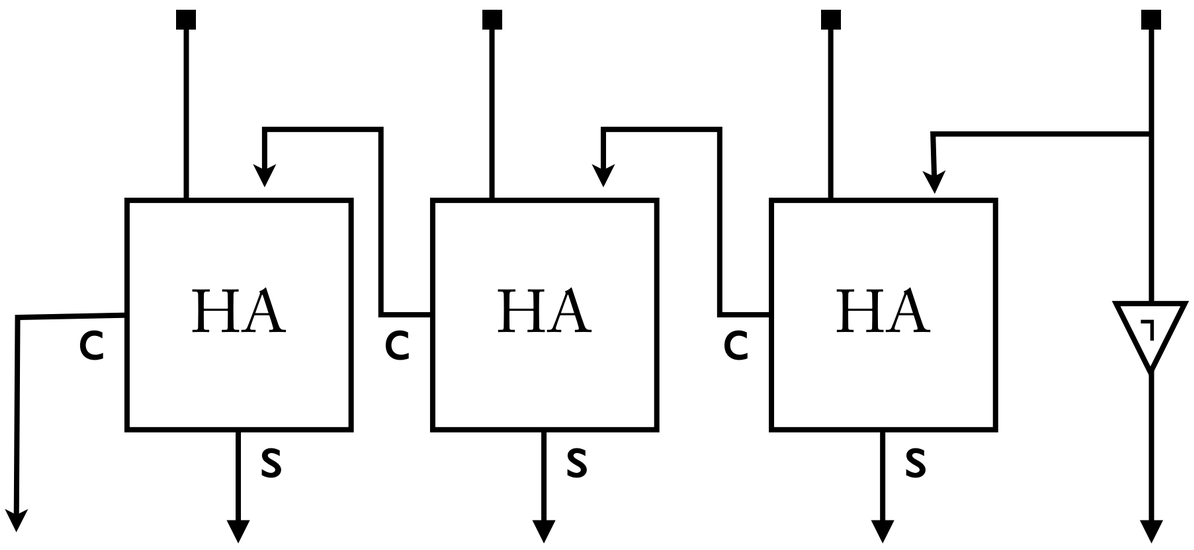
The CC just pushed each bit one position over in *space*. The SC pushes each bit one position over in *time*.



Binary +1

How do you compute Binary +1? Intuitively, by computing +1 on the first digit, outputting the sum, then adding the carry to the second digit, outputting the sum, then adding the carry to the third digit, outputting the sum, and so on…

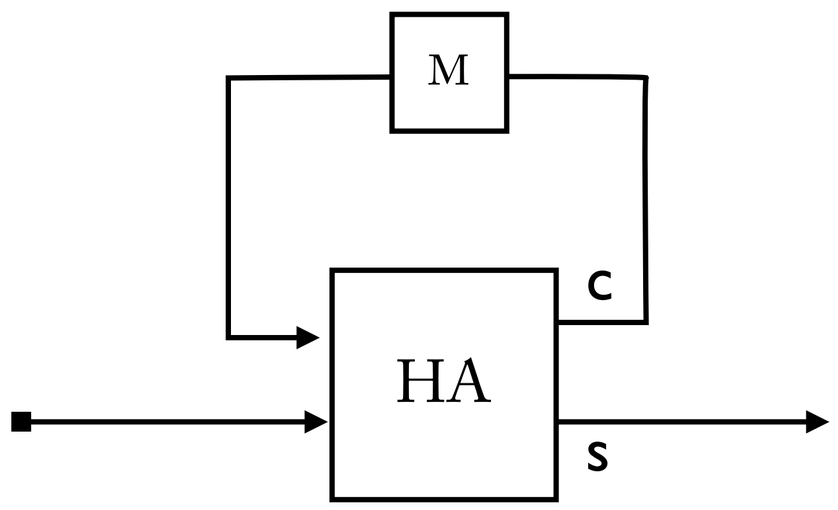
We've already developed a subcircuit for adding a carry to an input, outputting the sum, and computing the next carry. This is the half-adder! Before, we strung half-adders together *in space*, so that the carry from one was the input to the next.



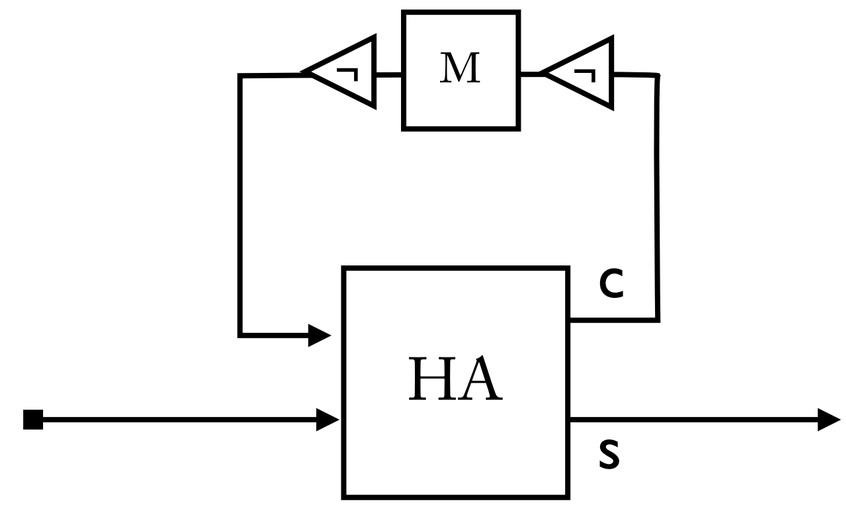
+1 B CC

Now we are going to string half-adders together in time. The idea is that, at each cycle, you store the carry digit in memory, and then use that memory as input on the next cycle. Something like the circuit at right.

The only problem with this is, it doesn't have any mechanism for adding *the first 1*. (Remember how we handled this with inverters for the combinatorial circuit?)



One way to solve this problem would be to start the memory off with a 1 in it— but then have it behave like a normal memory after that. Then the memory will push a 1 out on the first cycle, this will get inputted into the half-adder, and the result will be effectively adding 1 to your binary digit. This is exactly what a 1-first memory does. Here you go:



+1 B SC

Prove to yourself that this works! And meditate (deeply) on the relationship between this binary +1 machine, and our old combinatorial binary +1 machine. What one does in space, the other does in time!

In-Class Problems

Design SCs which solve the following problems.

1. **+2 T**

2. **+2 B**
   Challenge: what is the smallest circuit, in terms of numbers of logic gates, that you can think of that computes this function?

3. **2*x* B**

4. **2*x*+1 B**

5. **Come up with an SC that computes a two-place function.** Be creative— it doesn't have to be a familiar function; define your own if you like.

6. **Pattern Finder 2**
   Output = 1 as long as current digit and the previous 3 conform to the pattern "1100" or "0011".